

# Debugging and Program Inspection

Lecture 15

---

CS 1001

Kirk Duran

# Today

---

- ▶ Distinguish testing from debugging: tests detect mismatches; debugging investigates their causes.
- ▶ Use print statements as a simple tracing technique and identify where that technique stops scaling well.
- ▶ Use the VS Code visual debugger to pause execution without inserting diagnostic output into the program.
- ▶ Set breakpoints and inspect variables, expressions, and active call frames.
- ▶ Control execution using Continue, Step Over, Step Into, and Step Out.
- ▶ Use a hypothesis-driven process and a red–green cycle to make targeted corrections.

## Key idea

Debugging is not random code editing. It is the systematic comparison of expected execution with observed execution in order to locate the first meaningful divergence.

# Part 1: From a Failing Test to a Debugging Question



# Testing and Debugging Answer Different Questions

---

## Testing

Does the observed behavior satisfy the expected behavior?

- ▶ supplies inputs;
- ▶ states expectations;
- ▶ reports pass, failure, or error.

## Debugging

Why did execution produce the observed behavior?

- ▶ inspects control flow;
- ▶ inspects program state;
- ▶ narrows the location of the defect.

## Key idea

A failed test is evidence of a mismatch. It does not, by itself, identify the statement that caused the mismatch.

# A Symptom Is Not Necessarily the Defect

## Definition

A **symptom** is an observable incorrect result or failure. A **defect** is a fault in the program that contributes to that incorrect behavior.

## Example

```
def absolute_difference(a: int, b: int) -> int:
    if a > b:
        return a - b
    return a + b
```

## Observed symptom

`absolute_difference(4, 10)` returns 14, but the expected value is 6.

## Key idea

The incorrect returned value is where we notice the problem; the faulty expression occurs earlier in the execution path.

# Debugging Is Evidence-Based Program Inspection

---

## Definition

**Debugging** is the process of locating and correcting the cause of observed incorrect program behavior.

1. Reproduce a specific failure.
2. Predict what execution should do.
3. Observe what execution actually does.
4. Identify the earliest meaningful difference.
5. Correct the responsible code.
6. Run the tests again.

## Key idea

The purpose of a debugging tool is to increase the quality of our observations; the programmer still supplies the reasoning.

# Start from a Reproducible Case

---

## Implementation

```
def absolute_difference(a: int, b: int) -> int:
    if a > b:
        return a - b
    return a + b
```

## Tests

```
assert absolute_difference(10, 4) == 6  ✓
assert absolute_difference(4, 10) == 6  fails
```

## In class

Before inspecting the program, predict which conditional path the failing input should take and what expression should produce the final value.

## Part 2: Print Debugging

---

# Print Statements Can Expose Intermediate State

---

## Instrumented program

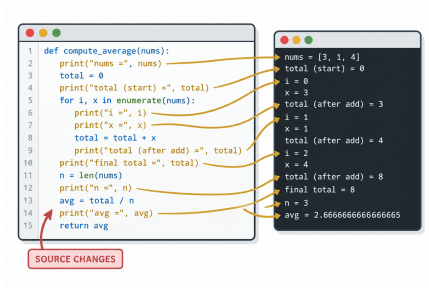
```
def final_price(price: float, rate: float) -> float:
    subtotal = price
    print("subtotal:", subtotal)
    tax = subtotal * rate
    print("tax:", tax)
    total = subtotal - tax
    print("total:", total)
    return total
```

## Key idea

Diagnostic prints turn selected internal values into visible output, allowing us to compare actual state with our predicted state.

# Print Debugging Requires Source-Code Changes

- ▶ We must decide what to print before running the program.
- ▶ Adding more observations requires editing the source code again.
- ▶ Diagnostic output can become mixed with the program's intended output.
- ▶ Temporary prints can be forgotten after the defect is corrected.



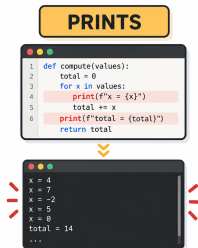
## Important

Print debugging is not inherently wrong. It is a useful technique with practical limits.

# From Instrumenting the Program to Inspecting the Program

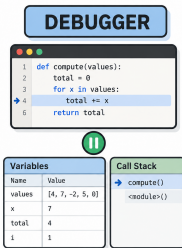
## Print debugging

Modify code to emit observations during execution.



## Visual debugger

Pause execution and inspect runtime state through the development environment.



## Part 3: The VS Code Visual Debugger

---

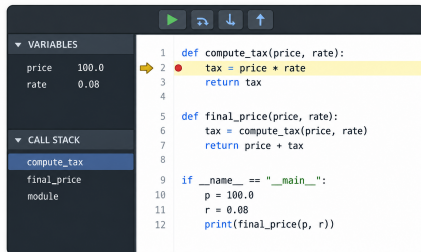
# A Debugger Executes the Program Under Observation

## Definition

A **debugger** is a development tool that can suspend a running program, expose runtime state, and provide controlled operations for resuming execution.

While execution is paused, we can inspect:

- ▶ the current source location;
- ▶ variable bindings;
- ▶ active function calls;
- ▶ selected expressions;
- ▶ breakpoints controlling future pauses.



# A Breakpoint Requests a Controlled Pause

## Definition

A **breakpoint** marks a source location at which the debugger should suspend normal execution when that executable location is reached.

## Example

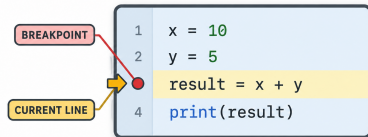
```
def final_price(price: float, rate: float) -> float:
    subtotal = price
    tax = subtotal * rate breakpoint
    total = subtotal + tax
    return total
```

## Key idea

When execution pauses at the breakpoint, inspect the state that exists before advancing through the suspicious computation.

# The Current Line Is a Control-Flow Location

- ▶ The debugger highlights the source location at which execution is currently suspended.
- ▶ The highlighted line identifies what is about to be executed or the current operation being stepped through, depending on the debugger state.
- ▶ Variables shown in the debugger represent state at that pause point.



## Tracing discipline

Inspect state *before* stepping, predict the effect of the next statement, then step and compare.

# The Variables View Shows Bindings in the Selected Frame

## Paused inside a call

```
def final_price(price: float, rate: float) -> float:  
    subtotal = price  
    tax = subtotal * rate  
    total = subtotal + tax
```

| Name     | Current value |
|----------|---------------|
| price    | 100.0         |
| rate     | 0.08          |
| subtotal | 100.0         |
| tax      | 8.0           |

## Key idea

The Variables view is the debugger equivalent of the binding tables we have been constructing manually.

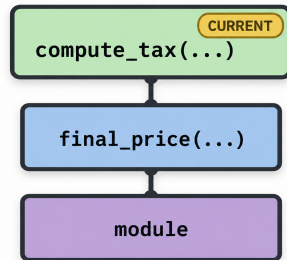
# The Call Stack Shows Active Function Calls

## Example execution

main code

```
↓ final_price(...)
  ↓ compute_tax(...)
```

While `compute_tax` is active, the earlier call to `final_price` has not finished.



## Key idea

Selecting a different frame lets us inspect the bindings associated with that active call.

# Watch Expressions Track Questions About State

---

## Definition

A **watch expression** is an expression the debugger reevaluates while execution is paused so the programmer can monitor its current value.

## Examples

```
subtotal * rate  
a > b  
result == expected
```

- ▶ Watches are useful when a derived value matters but is not stored in its own variable.
- ▶ Prefer simple observation expressions while learning; a debugger is most useful when it does not change the state being investigated.

# Step Over Executes the Current Line at the Current Abstraction Level

---

## Paused here

```
tax = compute_tax(subtotal, rate)
```

## Step Over

Execute the line, including the call to `compute_tax`, but do not pause on each statement inside that function.

## Afterward

Pause at the next statement in the current frame with `tax` now bound to the returned value.

## Key idea

Step Over is appropriate when the called function is not currently the object of investigation.

# Step Into Follows a Function Call

---

## Paused here

```
tax = compute_tax(subtotal, rate)
```

## Step Into

Enter the called function and suspend execution inside its body.

## New frame

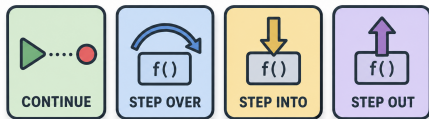
Inspect parameter bindings and local variables belonging to the `compute_tax` call.

## Key idea

Step Into crosses a function abstraction boundary so we can inspect the implementation of the call.

# The Four Core Execution Controls Serve Different Purposes

| Action    | Effect                                                            |
|-----------|-------------------------------------------------------------------|
| Continue  | Run until the next stopping event.                                |
| Step Over | Execute the current line without tracing into calls on that line. |
| Step Into | Follow a called function and inspect its execution.               |
| Step Out  | Finish the current function and pause in its caller.              |



## Part 4: Debugging as a Controlled Trace

---

# The Debugger Extends the Manual Trace Model

---

## Manual trace

- ▶ current statement;
- ▶ current bindings;
- ▶ active frame;
- ▶ predicted next state.

## Debugger

- ▶ highlighted source location;
- ▶ Variables view;
- ▶ Call Stack view;
- ▶ controlled stepping.

## Key idea

The debugger does not replace tracing. It lets us compare our trace model with the actual execution state one step at a time.

# Tracing Example: Find the First Incorrect State

## Buggy function

```
def final_price(price: float, rate: float) -> float:
    subtotal = price
    tax = subtotal * rate
    total = subtotal - tax
    return total
```

For `final_price(100.0, 0.08)`, suppose the specification expects 108.0.

| Pause point    | Expected state   | Observed state |
|----------------|------------------|----------------|
| after subtotal | subtotal = 100.0 | 100.0          |
| after tax      | tax = 8.0        | 8.0            |
| after total    | total = 108.0    | 92.0           |

## Key idea

The first state divergence occurs when `total` is computed, sharply narrowing the defect location.

# Place Breakpoints Near a Hypothesis, Not Everywhere

---

- ▶ A breakpoint before every line produces many pauses but little prioritization.
- ▶ A useful breakpoint is placed near the earliest location where the failing behavior could plausibly emerge.
- ▶ Begin before the suspicious computation so that the inputs to that computation can also be inspected.

## Example hypothesis

“The tax amount is correct, but the final price is combining subtotal and tax incorrectly.”

## Breakpoint choice

Pause on or immediately before `total = subtotal - tax`.

# Step Over When the Callee Is Not Suspect

---

## Program

```
def compute_tax(amount: float, rate: float) -> float:
    return amount * rate

def final_price(price: float, rate: float) -> float:
    tax = compute_tax(price, rate)
    return price - tax
```

If previous tests already establish that `compute_tax` behaves correctly, Step Over the call and inspect the returned value in `tax`.

## Key idea

Debugging is selective. Do not inspect an implementation merely because the debugger can enter it.

# Step Into When the Returned Value Is Already Wrong

## Suppose

After stepping over `tax = compute_tax(price, rate)`, the debugger shows `tax = -8.0` when `8.0` was expected.

1. Restart the failing case.
2. Pause before the call to `compute_tax`.
3. Verify the argument values.
4. Step Into the call.
5. Trace the callee's parameter bindings and expression evaluation.

## Key idea

Step Into is justified by evidence that the incorrect state exists at or before the function boundary.

# The Call Stack Answers “How Did We Get Here?”

---

Paused inside `compute_tax`

```
graph BT; A[compute_tax current frame] --> B[final_price caller]; B --> C[test_final_price earlier caller];
```

compute\_tax current frame  
↑  
final\_price caller  
↑  
test\_final\_price earlier caller

- ▶ Select `compute_tax` to inspect its parameters and locals.
- ▶ Select `final_price` to inspect the still-active caller frame.
- ▶ The call stack reconstructs the active chain of calls leading to the current location.

# Conditionals Should Be Debugged by Following One Concrete Path

---

## Buggy function

```
def shipping_cost(weight: float) -> float:  
    if weight < 10:  
        return 5.0  
    return 8.0
```

Suppose the specification says packages weighing *at most* 10 pounds cost 5.0.

## In class

For the failing input 10.0, identify the expected condition value, the observed condition value, and the first point where the execution path diverges.

## Key idea

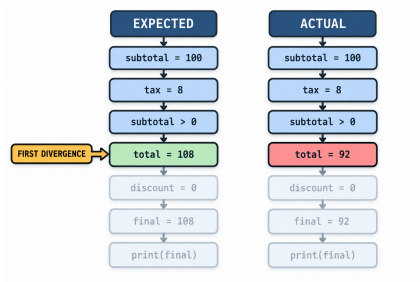
A conditional defect may produce the wrong branch even when every statement inside each branch is individually correct.

# The First Divergence Principle Narrows the Search

When debugging a concrete failing case:

1. predict the next relevant state;
2. observe the actual state;
3. advance only far enough to compare again;
4. stop broadening the search once the first incorrect state is found.

Later incorrect values may be consequences rather than independent defects.



## Part 5: Red–Green Development

---

# A Failing Test Gives Us the Red State

---

## RED

At least one test representing required behavior is currently failing.

- ▶ Red is not merely “the code is bad.”
- ▶ It identifies a specific observable behavior that the current implementation does not satisfy.
- ▶ The failure gives us a reproducible case from which debugging can begin.

## Key idea

Do not erase the evidence by weakening the test simply to make it pass. First determine whether the test's expectation correctly represents the specification.

# Debugging Should Produce a Targeted Change

---

## Before

```
total = subtotal - tax
```

## Evidence

```
subtotal = 100.0, tax = 8.0, expected total = 108.0, observed 92.0
```

## Targeted correction

```
total = subtotal + tax
```

## Key idea

The edit should follow from the diagnosis. Random changes make it difficult to know which change affected the behavior and why.

# Green Means the Relevant Tests Pass

---

## GREEN

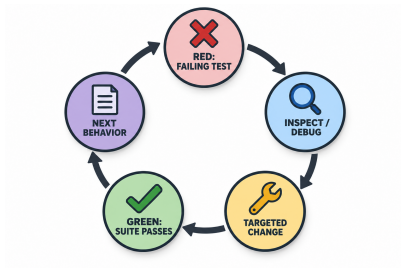
The corrected implementation now satisfies the current test suite.

- ▶ First rerun the test that exposed the defect.
- ▶ Then run the complete relevant test suite.
- ▶ A local correction can accidentally change behavior elsewhere.

## Key idea

Re-running existing tests after a change checks for regressions: previously expected behavior that no longer works after the modification.

# Red and Green Form a Development Feedback Loop



## Core loop

failing expectation → inspect execution → targeted change → passing tests

## Key idea

The cycle is useful because each code change has immediate executable feedback about whether the expected behavior was restored.

## Part 6: An Integrated Debugging Workflow

---

# A Practical Debugging Procedure

---

1. Run the tests and select one reproducible failure.
2. Read the expected and actual values carefully.
3. Predict the execution path for the failing input.
4. Form a specific hypothesis about where the first divergence may occur.
5. Set a breakpoint before that location.
6. Inspect state, then Step Over or Step Into deliberately.
7. Correct the defect supported by the evidence.
8. Rerun the failing test, then the full relevant suite.

# Integrated Example: Debugging Across a Function Call

## Implementation

```
def apply_discount(price: float, rate: float) -> float:
    return price + price * rate

def sale_price(price: float, rate: float) -> float:
    discounted = apply_discount(price, rate)
    return discounted
```

## Failing case

`sale_price(100.0, 0.20)` should return 80.0, but returns 120.0.

## In class

Where would you place the first breakpoint? Would you initially Step Over or Step Into `apply_discount`? What observation would justify changing that decision?

# Common Debugging Mistakes

---

- ▶ Editing several unrelated statements before rerunning the test.
- ▶ Stepping through every line without a hypothesis about what information is needed.
- ▶ Looking only at the final incorrect value rather than finding where state first became incorrect.
- ▶ Stepping into every function call regardless of whether evidence implicates that function.
- ▶ Fixing the one failing case but not rerunning the rest of the tests.
- ▶ Leaving diagnostic code in place after it no longer serves a purpose.

## Key idea

Effective debugging reduces uncertainty systematically; ineffective debugging often creates more uncontrolled changes than evidence.

# Final Trace: Locate the Defect Before Editing

## Program

```
def is_eligible(age: int) -> bool:
    return age > 18

def admission(age: int, has_ticket: bool) -> str:
    if is_eligible(age) and has_ticket:
        return "admit"
    return "deny"
```

Specification: age 18 is eligible when the person also has a ticket.

## In class

For `admission(18, True)`, identify: (1) the expected result, (2) the first useful breakpoint, (3) where Step Into becomes useful, and (4) the first observed state or expression that would differ from the specification.

# Takeaways

---

- ▶ Tests identify reproducible mismatches between expected and actual behavior.
- ▶ Print statements can reveal state, but visual debugging scales better when detailed runtime inspection is needed.
- ▶ Breakpoints pause execution so variables, expressions, and active call frames can be inspected directly.
- ▶ Continue, Step Over, Step Into, and Step Out intentionally control the level and amount of observation.
- ▶ Debugging should seek the first meaningful divergence between predicted and observed execution.
- ▶ Red–green development connects failing tests, evidence-based corrections, and repeated verification.

## Key idea

Tests tell us *that* behavior is wrong. Debugging helps us explain *why* it is wrong.